

Lab Report

Database Design Lab

DDL · DML · Constraints · Grouping & Aggregation

Prepared for

Md Nasim Adnan

Assistant Professor

CSE 364.7 — Database Design Lab

Prepared by

Abid Hasan

ID	:	2023200000720
Batch	:	65
Section	:	07
Date	:	11 June 2026



Southeast University

Department of Computer Science and engineering

TABLE OF CONTENTS

1. Introduction and Objectives.....	1
2.Schema Overview.....	2
3. Data Definition Language (DDL) & Table Creation.....	4
4. Constraint Demonstration & Validation.....	7
5. Data Manipulation Language.....	13
6. Logical Filtering with Where.....	19
7. Aggregation and Group By.....	21
8. Database State Summary.....	25
9. Conclusion.....	26

1. Introduction and Objectives

In relational database designing, maintaining data integrity, consistency, and reliability is a fundamental requirement. While application-level validation plays an important role, robust database systems enforce critical business rules directly at the storage layer to prevent invalid or inconsistent data from entering the system. The primary objective of this laboratory series is to bridge the gap between conceptual database design and practical implementation by developing structured relational databases using MySQL.

This report presents the complete workflow of database development, beginning with schema creation and constraint enforcement through Data Definition Language (DDL) commands. It further explores advanced data manipulation using Data Manipulation Language (DML), including record insertion, updates, deletions, filtering, sorting, various types of data selection, and grouping operations. Various integrity constraints such as PRIMARY KEY, FOREIGN KEY, UNIQUE, NOT NULL, and CHECK constraints are utilized to ensure data accuracy and maintain referential integrity.

Through hands-on experimentation in MySQL, this laboratory exercise demonstrates how relational database systems provide a secure and consistent environment for storing and managing data. By embedding validation rules directly into the database layer, erroneous or malformed data can be prevented from compromising system reliability and downstream applications.

All SQL operations documented in this report were executed on a live MySQL environment under the guidance of Faculty MD. NASIM ADNAN.

2. Schema Overview

The system modelled across all lab exercises is an Employee Management System for an academic institution. It tracks employees across multiple categories — teachers, officers, and managers — each with their own attributes, while sharing a common Employee base record. Department and Designation serve as reference lookup tables.

2.1 Entity Summary & Relationships

Table	Role	Key Relationships
Employee	Central entity — stores all common employee data	References Department, Designation
Department	Lookup — stores department names	Referenced by Employee, Manager
Designation	Lookup — stores designation titles	Referenced by Employee
Teacher_Details	Subtype of Employee — stores academic-specific info	Employee (Ids) via FK
Officer_Details	Subtype of Employee — stores admin-specific info	Employee (Ids) via FK
Manager	Independent manager table with own attributes	References Department

2.2 Schema Diagram Reference

The diagram below (from the MySQL Workbench EER model) shows all six tables with their columns, data types, and relationships. Crow's-foot notation indicates cardinality: Employees can have one Teacher_Details or Officer_Details record (1:1 subtype). Multiple employees belong to one Department (N:1).

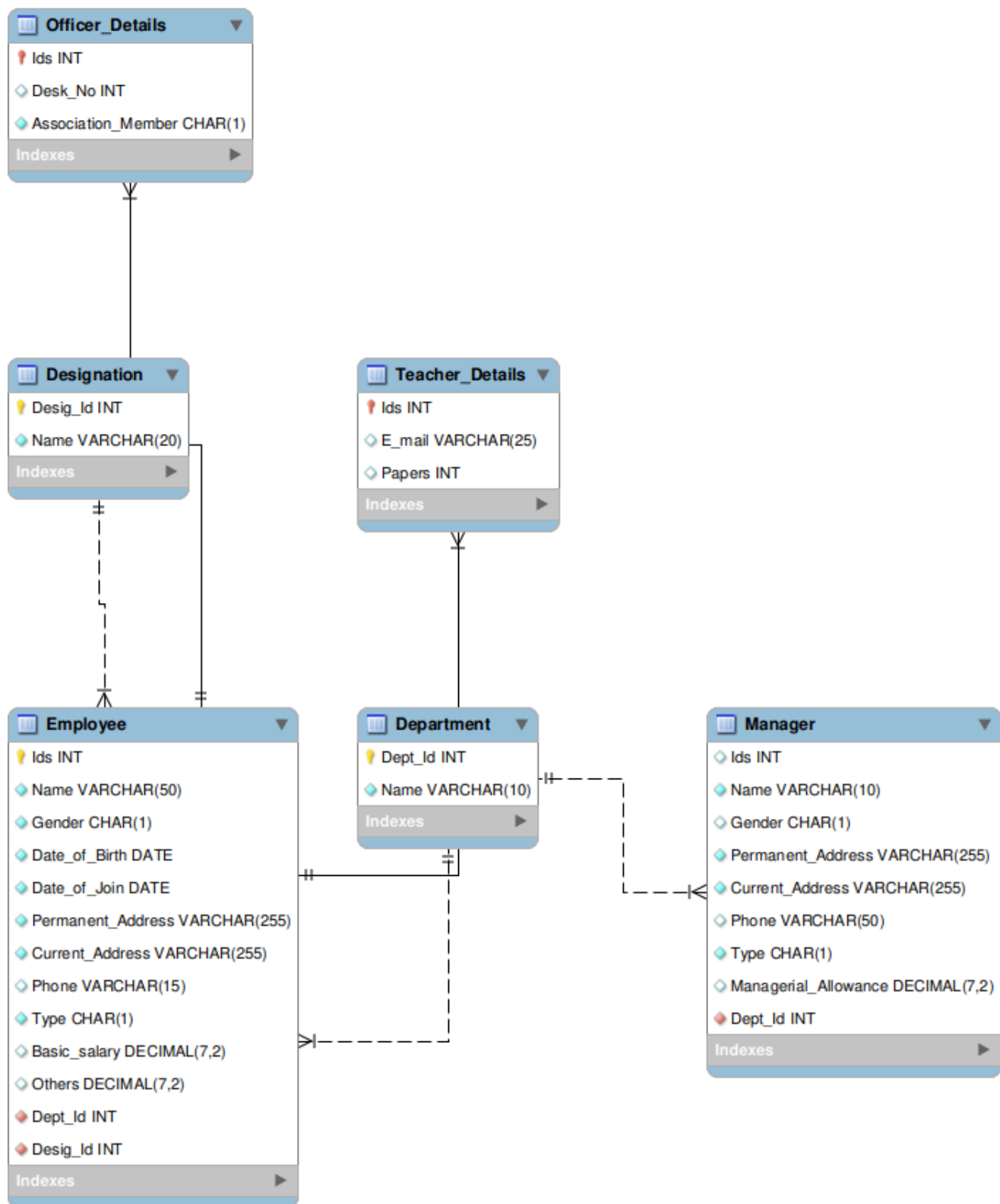


Fig : Reference Schema

3. Data Definition Language (DDL) & Table Creation

DDL is the subset of SQL used to define and manage the structure of a database tables, columns, constraints, and relationships. Unlike DML (Data Manipulation Language), DDL statements alter the schema itself rather than the data within it. Every table used in the lab was built using the DDL steps described in this section.

3.1 Overview of All Constraint Types

Before examining the individual table definitions, the following reference table summarises all five standard SQL constraint types used in this lab. Each constraint enforces a different dimension of data integrity:

Constraint Type	SQL Keyword	What It Enforces	Example in EHRAMS
Entity Integrity	PRIMARY KEY	Each row is uniquely identifiable; no duplicates, no NULL in key column	Employee(Ids), Department(Dept_Id)
Referential Integrity	FOREIGN KEY	A value in one table must exist as a primary key in the parent table	Employee.Dept_Id → Department.Dept_Id
Domain Constraint	CHECK	Column value must satisfy a boolean expression	Gender IN ('M','F'), Basic_salary >= 5000
Attribute Constraint	NOT NULL	Column must always have a value; NULL is forbidden	Name, Gender, Date_of_Birth, Type
Uniqueness	UNIQUE	All values in the column must be distinct (NULLs are allowed)	Officer_Details.Desk_No

3.2 Lookup Tables: Department & Designation

Lookup tables are created first so their primary keys are ready to be referenced as foreign keys by other tables.

```
-- Create lookup table for Departments
```

```
CREATE TABLE Department (Dept_Id INT(5),Name VARCHAR(10) NOT NULL, CONSTRAINT  
Pk_Ids_Department PRIMARY KEY (Dept_Id));
```

```
-- Create lookup table for Designations
```

```
CREATE TABLE Designation (Desig_Id INT(5),Name VARCHAR(20) NOT NULL, CONSTRAINT  
Pk_Ids_Designation PRIMARY KEY (Desig_Id));
```

Each lookup table is assigned a named primary key constraint. Naming constraints (e.g., Pk_Ids_Department) makes error messages more readable and simplifies schema maintenance.

3.3 Base Employee Table with Constraints

The Employee table is the core entity that holds demographic, employment, and financial data. Multiple constraint types are applied at creation time:

```
CREATE TABLE Employee (  
  Ids INT(5),  
  Name VARCHAR(50) NOT NULL,  
  Gender CHAR(1) NOT NULL,  
  Date_of_Birth DATE NOT NULL,  
  Date_of_Join DATE NOT NULL,  
  Permanent_Address VARCHAR(255) NOT NULL,  
  Current_Address VARCHAR(255) NOT NULL,  
  Phone VARCHAR(15),  
  Type CHAR(1) NOT NULL,  
  Basic_salary DECIMAL(7,2),  
  Others DECIMAL(7,2),  
  
  CONSTRAINT Pk_Employee PRIMARY KEY (Ids),  
  CONSTRAINT Chk_Gender CHECK (Gender IN ('M','F')),  
  CONSTRAINT Chk_EmpType CHECK (Type IN ('T','O','S')),  
  CONSTRAINT Chk_BasicSalary CHECK (Basic_salary >= 5000),  
  CONSTRAINT Chk_Others CHECK (Others >= 1500)  
);
```

3.4 Subclass Tables: Teacher_Details & Officer_Details

These tables follow a table-per-subtype inheritance pattern. Each shares its primary key with Employee which is simultaneously the foreign key back to the parent table.

Multiple constraint types are applied at creation time:

```
-- Teacher subclass
CREATE TABLE Teacher_Details (
  Ids   INT(5),
  E_mail VARCHAR(25),
  Papers INT(4),
  CONSTRAINT Pk_Teacher PRIMARY KEY (Ids),
  CONSTRAINT Chk_Papers CHECK (Papers >= 0)
);
ALTER TABLE Teacher_Details
  ADD CONSTRAINT Fk_Teacher_Employee
  FOREIGN KEY (Ids) REFERENCES Employee(Ids);

-- Officer subclass
CREATE TABLE Officer_Details (
  Ids      INT(5),
  Desk_No   INT(3) UNIQUE,
  Association_Member CHAR(1) NOT NULL,
  CONSTRAINT Pk_Officer PRIMARY KEY (Ids),
  CONSTRAINT Chk_AssocMbr CHECK (Association_Member IN ('Y','N'))
);
ALTER TABLE Officer_Details
  ADD CONSTRAINT Fk_Officer_Employee
  FOREIGN KEY (Ids) REFERENCES Employee(Ids);
```

3.5 Manager Table (for JOIN Practise)

The Manager table is an independent entity visible in the schema. It stores managerial staff who are not necessarily in the Employee table. It demonstrates all major constraint types in a single definition.

```
CREATE TABLE Manager (
  Ids INT(5),
  Name VARCHAR(10) NOT NULL,
  Gender CHAR(1) NOT NULL,
  Permanent_Address VARCHAR(255) NOT NULL,
  Current_Address VARCHAR(255) NOT NULL,
```



```

Phone VARCHAR(50) UNIQUE,
Type CHAR(1) NOT NULL,
Managerial_Allowance DECIMAL(7,2),
Dept_Id INT(5) NOT NULL,

-- Entity Integrity
CONSTRAINT Pk_Manager PRIMARY KEY (Ids),

-- Domain Constraints
CONSTRAINT Chk_Mgr_Gender CHECK (Gender IN ('M','F')),
CONSTRAINT Chk_Mgr_Type CHECK (Type IN ('T','O','S')),
CONSTRAINT Chk_Mgr_Allowance CHECK (Managerial_Allowance >= 3000),

-- Referential Integrity
CONSTRAINT Fk_Manager_Dept
    FOREIGN KEY (Dept_Id) REFERENCES Department(Dept_Id)
);

```

The Manager table references the Department via Dept_Id. Department rows must be inserted before any Manager rows, otherwise the foreign key constraint will reject the insert.

4. Constraint Demonstration: Course_Enrollment Table

To illustrate all five constraint types working together in a single table, we introduce a new table Course_Enrollment that was not part of the original lab schema but is contextually appropriate for a university HR/academic system. This table records which teachers are enrolled in which professional development courses.

Note : This is a new table created specifically to validate constraint behaviour. It is appropriate to the system context and exercises every constraint type in one definition.

4.1 Table Definition — All Five Constraint Types

```

-- — New table: Course_Enrollment

```

```

-- Demonstrates all 5 SQL constraint types in one definition

CREATE TABLE Course_Enrollment (

```

```

Enrollment_Id INT(5),
Teacher_Ids INT(5) NOT NULL,           -- Attribute Constraint (NOT NULL)
Course_Code VARCHAR(10) NOT NULL,      -- Attribute Constraint (NOT NULL)
Course_Title VARCHAR(100) NOT NULL,    -- Attribute Constraint (NOT NULL)
Enrollment_Date DATE NOT NULL,         -- Attribute Constraint (NOT NULL)
Completion_Date DATE,                  -- nullable: may not be completed yet
Grade CHAR(2),
Credit_Hours INT(2),
Fee DECIMAL(8,2),

```

-- 1) Entity Integrity

```
CONSTRAINT Pk_Enrollment_Id PRIMARY KEY (Enrollment_Id),
```

-- 2) Uniqueness: a teacher cannot enroll in the same course twice

```
CONSTRAINT Uk_Teacher_Course UNIQUE (Teacher_Ids, Course_Code),
```

-- 3) Referential Integrity: teacher must exist in Employee

```
CONSTRAINT Fk_CE_Teacher FOREIGN KEY (Teacher_Ids) REFERENCES Employee(Ids),
```

-- 4) Domain Constraints

```
CONSTRAINT Chk_Grade CHECK (Grade IN ('A+', 'A', 'A-', 'B+', 'B', 'B-', 'C', 'F')),
```

```
CONSTRAINT Chk_Credit_Hours CHECK (Credit_Hours BETWEEN 1 AND 6),
```

```
CONSTRAINT Chk_Fee CHECK (Fee >= 0),
```

```
CONSTRAINT Chk_CE_Timeline CHECK (Completion_Date IS NULL
                                OR Completion_Date >= Enrollment_Date)
```

```
);
```

4.2 INSERT Examples — Valid Records

The following inserts succeed because every value satisfies every active constraint:

```
-- — Valid inserts
```

```
INSERT INTO Course_Enrollment
(Enrollment_Id, Teacher_Ids, Course_Code, Course_Title,
 Enrollment_Date, Completion_Date, Grade, Credit_Hours, Fee)
VALUES
(1, 1, 'ML-401', 'Machine Learning Foundations',
 '2025-01-10', '2025-06-15', 'A+', 3, 5000.00);

INSERT INTO Course_Enrollment
(Enrollment_Id, Teacher_Ids, Course_Code, Course_Title,
 Enrollment_Date, Completion_Date, Grade, Credit_Hours, Fee)
VALUES
(2, 2, 'DB-301', 'Advanced Database Systems',
 '2025-02-01', '2025-07-01', 'A', 3, 4500.00);

INSERT INTO Course_Enrollment
(Enrollment_Id, Teacher_Ids, Course_Code, Course_Title,
 Enrollment_Date, Completion_Date, Grade, Credit_Hours, Fee)
VALUES
(3, 5, 'NET-501', 'Computer Networks Advanced',
 '2025-03-15', NULL, NULL, 4, 6000.00); -- still in progress: NULL Completion_Date & Grade OK

INSERT INTO Course_Enrollment
(Enrollment_Id, Teacher_Ids, Course_Code, Course_Title,
 Enrollment_Date, Completion_Date, Grade, Credit_Hours, Fee)
VALUES
(4, 3, 'ML-401', 'Machine Learning Foundations',
 '2025-01-10', '2025-06-20', 'B+', 3, 5000.00); -- Teacher_Ids=3, same course as row 1: OK
```

All four inserts above pass validation: Enrollment_Ids are unique, Teacher_Ids exist in Employee, Grades are in the allowed set, Credit_Hours are within 1-6, Fees are non-negative, and no Completion_Date precedes its Enrollment_Date.

See the resulting table:

```
select * from Course_Enrollment;
```

Enrollment_Id	Teacher_Ids	Course_Code	Course_Title	Enrollment_Date	Completion_Date	Grade	Credit_Hours	Fee
1	1	ML-401	Machine Learning Foundations	2025-01-10	2025-06-15	A+	3	5000
2	2	DB-301	Advanced Database Systems	2025-02-01	2025-07-01	A	3	4500
3	5	NET-501	Computer Networks Advanced	2025-03-15			4	6000
4	3	ML-401	Machine Learning Foundations	2025-01-10	2025-06-20	B+	3	5000

4.3 ALTER TABLE on Course_Enrollment

After the table is live, we add a new column and a new constraint dynamically using ALTER TABLE:

```
-- — Add a Department column and link it to Department table —————
```

```
ALTER TABLE Course_Enrollment
```

```
ADD COLUMN Dept_Id INT(5);
```

```
ALTER TABLE Course_Enrollment
```

```
ADD CONSTRAINT Fk_CE_Dept
```

```
FOREIGN KEY (Dept_Id) REFERENCES Department(Dept_Id);
```

```
-- — Update existing rows with Department data —————
```

```
UPDATE Course_Enrollment SET Dept_Id = 1 WHERE Enrollment_Id IN (1,4);
```

```
UPDATE Course_Enrollment SET Dept_Id = 2 WHERE Enrollment_Id = 2;
```

```
UPDATE Course_Enrollment SET Dept_Id = 1 WHERE Enrollment_Id = 3;
```

4.4 Constraint Violation Scenarios

Each scenario below demonstrates a different constraint being enforced and the exact error MySQL returns.

Scenario A — PRIMARY KEY Violation (Entity Integrity)

```
-- Enrollment_Id = 1 already exists
INSERT INTO Course_Enrollment
(Enrollment_Id, Teacher_Ids, Course_Code, Course_Title, Enrollment_Date, Credit_Hours, Fee)
VALUES (1, 6, 'AI-601', 'Artificial Intelligence', '2025-04-01', 3, 4000.00);
```

Error Code: 1062. Duplicate entry '1' for key 'Course_Enrollment.PRIMARY'

The database scans the primary key B-Tree, finds Enrollment_Id = 1 already mapped to an existing record, and rejects the insert entirely. The existing row is untouched.

Scenario B — UNIQUE Constraint Violation

```
-- Enrollment_Id = 1 already exists
INSERT INTO Course_Enrollment
(Enrollment_Id, Teacher_Ids, Course_Code, Course_Title, Enrollment_Date, Credit_Hours, Fee)
VALUES (1, 6, 'AI-601', 'Artificial Intelligence', '2025-04-01', 3, 4000.00);
```

Error Code: 1062. Duplicate entry '1-ML-401' for key 'Course_Enrollment.Uk_Teacher_Course'

The composite UNIQUE constraint (Teacher_Ids, Course_Code) prevents the same teacher from enrolling in the same course twice. The combination (1, 'ML-401') already exists from the first valid insert.

Scenario C — CHECK Constraint Violations

```
-- Violation 1: Grade 'Z' is not in the allowed set
INSERT INTO Course_Enrollment
(Enrollment_Id, Teacher_Ids, Course_Code, Course_Title, Enrollment_Date, Grade, Credit_Hours, Fee)
VALUES (6, 6, 'OS-201', 'Operating Systems', '2025-04-01', 'Z', 3, 3000.00);
```

Error Code: 3819. Check constraint 'Chk_Grade' is violated.

```
-- Violation 2: Credit_Hours = 9 exceeds maximum of 6
INSERT INTO Course_Enrollment
(Enrollment_Id, Teacher_Ids, Course_Code, Course_Title, Enrollment_Date, Credit_Hours, Fee)
VALUES (7, 7, 'DS-701', 'Data Structures Advanced', '2025-04-01', NULL, 9, 2000.00);
```

Error Code: 3819. Check constraint 'Chk_Credit_Hours' is violated.

```
-- Violation 2: Credit_Hours = 9 exceeds maximum of 6
INSERT INTO Course_Enrollment
(Enrollment_Id, Teacher_Ids, Course_Code, Course_Title, Enrollment_Date, Credit_Hours, Fee)
VALUES (7, 7, 'DS-701', 'Data Structures Advanced', '2025-04-01', NULL, 9, 2000.00);
```

Error Code: 3819. Check constraint 'Chk_CE_Timeline' is violated.

In all CHECK violation cases, the engine evaluates the constraint expression against the incoming value before any disk write occurs. If the expression evaluates to FALSE, the row is discarded and the transaction is aborted.

Scenario D — FOREIGN KEY Violation (Referential Integrity)

```
-- Teacher_Ids = 999 does not exist in Employee table
INSERT INTO Course_Enrollment
(Enrollment_Id, Teacher_Ids, Course_Code, Course_Title, Enrollment_Date, Credit_Hours, Fee)
VALUES (9, 999, 'CV-901', 'Computer Vision', '2025-05-01', 3, 5500.00);
```

Error Code: 1452. Cannot add or update a child row: a foreign key constraint fails ('labDB`.`Course_Enrollment`, CONSTRAINT `Fk_CE_Teacher` FOREIGN KEY (`Teacher_Ids`) REFERENCES `Employee` (`Ids`))

The engine performs a lookup on Employee for Ids = 999. Since no such record exists, the insert is blocked. This prevents enrollments from being linked to ghost teachers that do not exist in the system.

Scenario E — NOT NULL Violation (Attribute Constraint)

```
-- Teacher_Ids = 999 does not exist in Employee table
INSERT INTO Course_Enrollment
(Enrollment_Id, Teacher_Ids, Course_Code, Course_Title, Enrollment_Date, Credit_Hours, Fee)
VALUES (9, 999, 'CV-901', 'Computer Vision', '2025-05-01', 3, 5500.00);
```

Error Code: 1048. Column 'Course_Title' cannot be null.

NOT NULL is enforced at the column level before any other constraint check. A NULL in a mandatory field is rejected immediately.

Constraint	Error Code	Trigger	Engine Action
PRIMARY KEY	ERROR 1062	Duplicate key value	Full transaction rollback
UNIQUE	ERROR 1062	Duplicate in unique column	Full transaction rollback
CHECK	ERROR 3819	Boolean expression = FALSE	Row discarded before write
FOREIGN KEY	ERROR 1452	Parent key not found	Insert/update blocked
NOT NULL	ERROR 1048	NULL supplied to NOT NULL col	Rejected at column parse stage

5. Data Manipulation Language (DML) — Advanced Operations

DML is the subset of SQL that operates on the **data inside** the tables (not the structure). Unlike DDL which builds the container, DML fills, modifies, and removes the contents. There are four core DML operations:

Operation	Statement	Purpose
Create data	INSERT	Add new rows into a table
Read data	SELECT	Retrieve and display rows
Modify data	UPDATE	Change values in existing rows
Remove data	DELETE	Remove rows from a table

5.1 INSERT —Adding Rows

INSERT places new records into a table. Every value must pass all active constraints — if any constraint fails, the entire row is discarded.

Syntax:

```
-- sql
INSERT INTO table_name (col1, col2, ...) VALUES (val1, val2, ...);
```

Single row insert:

```
-- sql
-- Insert one new employee
INSERT INTO Employee
(Ids, Name, Gender, Date_of_Birth, Date_of_Join,
Permanent_Address, Current_Address, Phone, Type,
Basic_salary, Others, Dept_Id, Desig_Id)
VALUES
(8, 'Tamal', 'M', '1985-03-10', '2010-06-01',
'Dhaka', 'Dhaka', '01812345678', 'T',
18000.00, 3000.00, 1, 4);
```

Multi-row insert (batch):

```
-- sql
-- Insert multiple departments in one statement
INSERT INTO Department VALUES
(5, 'Law'),
(6, 'Pharmacy');
```


Insert into Teacher_Details table:

```
-- sql
-- Insert Teacher_Details for the new employee (Ids=8 must exist in Employee first)
INSERT INTO Teacher_Details (Ids, E_mail, Papers)
VALUES (8, 'tamal@abc.ac.bd', 3);
```

5.2 UPDATE — Modifying Existing Rows

UPDATE changes column values in rows that already exist. Without a WHERE clause, **every row in the table is updated** — always include a WHERE unless a bulk update is intentional.

Syntax:

```
-- sql
UPDATE table_name SET col1 = val1, col2 = val2 WHERE condition;
```

Single column update:

```
-- sql
-- Give employee Ids=8 a salary raise
UPDATE Employee
SET Basic_salary = 20000.00
WHERE Ids = 8;
```

Multiple columns at once:

```
-- sql
-- Reassign Tamal to Department 2 and update his phone
UPDATE Employee
SET Dept_Id = 2,
Phone = '01987654321'
```

```
WHERE Ids = 8;
```

Update using arithmetic:

```
-- sql
-- Give a 10% basic salary increment to all CSE teachers
UPDATE Employee
SET Basic_salary = Basic_salary * 1.10
WHERE Dept_Id = 1 AND Type = 'T';
```

Update on Course_Enrollment:

```
-- sql
-- Mark enrollment 3 as completed with a grade
UPDATE Course_Enrollment
SET Completion_Date = '2025-09-01',
Grade = 'A-'
WHERE Enrollment_Id = 3;
```

MySQL's safe-update mode (`SQL_SAFE_UPDATES = 1`) blocks updates without a `WHERE` on an indexed column. In the lab we disable it with `SET SQL_SAFE_UPDATES = 0`; when doing bulk updates intentionally.

5.3 DELETE — Removing Rows

`DELETE` removes entire rows from a table. Like `UPDATE`, a missing `WHERE` clause deletes **all rows** in the table (the table structure survives — only data is removed).

Syntax:

```
-- sql
DELETE FROM table_name WHERE condition
```

Delete a specific row:

```
-- sql
-- Remove the newly added employee Ids=8
```

```
-- Must delete subclass row first (child before parent — FK order)
DELETE FROM Teacher_Details WHERE Ids = 8;
DELETE FROM Employee WHERE Ids = 8;
```

Delete with condition:

```
-- sql
sql
-- Remove all course enrollments with a failing grade
DELETE FROM Course_Enrollment
WHERE Grade = 'F';
```

Delete all rows (keep structure):

```
-- sql
-- Clear the Manager table entirely before re-inserting fresh data
DELETE FROM Manager;
```

Deletion order with foreign keys: You must delete child rows **before** parent rows. Attempting to delete an Employee row while a Teacher_Details row still references it produces ERROR 1451 a foreign key constraint violation in the reverse direction (parent-delete blocked by child).

5.4 SELECT with String Functions

String functions normalise, format, and analyse textual data essential for report generation and data cleaning.

```
-- sql
select name,
       upper(name),
       substring(name, 1, 3)
from employee;
```

Output table:

name	upper(name)	substring(name, 1, 3)
Nasim	NASIM	Nas
Arshad	ARSHAD	Ars

Fahima	FAHIMA	Fah
Nowrin	NOWRIN	Now
Rafiq	RAFIQ	Raf
Salam	SALAM	Sal
Zia	ZIA	Zia

Function	Effect
UPPER(Name)	Converts to uppercase — used for case-insensitive comparison
SUBSTRING(Name, 1, 3)	Extracts first 3 characters — useful for abbreviations
CHAR_LENGTH(Name)	Returns character count — used in min/max length queries

5.5 SELECT with Mathematical & Financial Functions

Financial calculations require precise decimal control to prevent rounding errors across mass reporting pipelines.

```
-- sql
select basic_salary,
       round(basic_salary / 3, 2) as split_salary_rounded,
       truncate(basic_salary / 3, 2) as split_salary_truncated,
       ceil(basic_salary / 7) as upper_integer_salary,
       floor(basic_salary / 7) as lower_integer_salary,
from employee;
```

Output table:

basic_salary	split_salary_rounded	split_salary_truncated	upper_integer_salary	lower_integer_salary
15000	5000	5000	2143	2142
12000	4000	4000	1715	1714
12000	4000	4000	1715	1714
10000	3333.33	3333.33	1429	1428
45000	15000	15000	6429	6428
20000	6666.67	6666.66	2858	2857

15000	5000	5000	2143	2142
-------	------	------	------	------

Function	Behaviour
ROUND(x, 2)	Rounds to 2 decimal places using standard mathematical rules
TRUNCATE(x, 2)	Chops off beyond 2 decimal places — never rounds up
CEIL(x)	Smallest integer $\geq x$
FLOOR(x)	Largest integer $\leq x$

6. Logical Filtering with WHERE

The WHERE clause filters individual rows before any grouping or aggregation occurs. MySQL supports a rich set of comparison, range, pattern, and set operators that can be combined with AND / OR / NOT for precise data slicing.

6.1 Filtering Operators Reference

Operator	Syntax	Meaning
=	Gender = 'M'	Exact equality match
BETWEEN	salary BETWEEN 12000 AND 60000	Inclusive range check on a column
IN	Address IN ('Khulna','Magura')	Tests membership against a set of values
NOT IN	Address NOT IN (...)	Excludes all values in the given set
LIKE '%a%'	Name LIKE '%a%'	Pattern match: % = any sequence, _ = one char
NOT LIKE	Phone NOT LIKE '011%'	Excludes rows matching the pattern
AND / OR	cond1 AND cond2	Logical combination of multiple conditions
ORDER BY	ORDER BY salary DESC, name ASC	Multi-column sort of the final result set

6.2 Filtering Queries

Query: Employees from specific cities with salary range and name pattern

```
-- sql
select Name,Gender,Date_of_Birth,Date_of_Join,Permanent_Address,
```

```
Current_Address,Phone,Type,Basic_salary,Others from Employee
where permanent_address in ('Khulna','Magura','Sylhet')
and basic_salary between 12000.00 and 60000.00
and name like '%a%'
and phone not like '011%'
order by basic_salary desc, name asc;
```

Output table:

Name	Gender	Date_of_Birth	Date_of_Join	Permanent_Address	Current_Address	Phone	Type	Basic_salary	Others
Rafiq	M	1950-12-01	1991-01-01	Khulna	Dhaka	1914684384	T	45000	15000
Nasim	M	1979-12-17	2001-01-01	Magura	Dhaka	1730016854	T	15000	5000
Fahima	F	1975-01-01	1999-01-01	Sylhet	Dhaka	1819750750	T	12000	3500

Query: Employees hired before 2003 (date range filter)

```
-- sql
select name, date_of_join, basic_salary from employee
where date_of_join between '1990-01-01' and '2003-01-01'
order by date_of_join;
```

Output Table:

name	date_of_join	basic_salary
Rafiq	1991-01-01	45000
Salam	1995-01-01	20000
Fahima	1999-01-01	12000
Zia	1999-01-01	15000
Nasim	2001-01-01	15000
Nowrin	2002-01-01	10000

Query: Teachers earning above Nasim's salary (subquery in WHERE)

```
-- sql
```

```
select Name,Basic_salary from employee
where basic_salary > (select basic_salary from employee where ids = 1);
```

Output table:

Name	Basic_salary
Rafiq	45000
Salam	20000

7. Aggregation & GROUP BY

Aggregation functions collapse multiple rows into a single summarised value. They transform raw row-level data into operational insights across groups defined by GROUP BY.

7.1 Core Aggregation Functions

Function	Applied To	Result
COUNT(*)	All rows in group	Total number of records in the group
MAX(col)	Numeric or text	Largest value within the group
MIN(col)	Numeric or text	Smallest value within the group
SUM(col)	Numeric column	Arithmetic sum of all values in the group
AVG(col)	Numeric column	Arithmetic mean of all values in the group

7.2 Single-Column Aggregations

Basic aggregation queries on Employee table

```
select count(*) from employee;
```

Output : 7

```
select count(distinct dept_id) from employee;
```

Output : 3

```
select max(length(name)) from employee;
```

Output : 6

```
select min(length(name)) from employee;
```

Output : 3

```
select avg(basic_salary) from employee;
```

Output : 18428.571429

```
select avg(basic_salary) from employee where dept_id = 3;
```

Output : 13500.000000

7.3 GROUP BY — Single Column

Average salary per department

```
-- sql
select dept_id, avg(basic_salary) as avg_basic_salary
from employee
group by dept_id
order by avg_basic_salary;
```

Output table:

dept_id	avg_basic_salary
3	13500
2	16000
1	23333.33333

Average salary per department

```
-- sql
select dept_id, avg(basic_salary) as avg_basic_salary
from employee
group by dept_id
order by avg_basic_salary;
```

Output table:

dept_id	avg_basic_salary
3	13500
2	16000
1	23333.33333

Total salary (basic + others) per department

```
-- sql
select d.name, avg(e.basic_salary + e.others) as avg_sal
from employee e, department d
```



```
where e.dept_id = d.dept_id  
group by d.name;
```

Output table:

name	avg_sal
CSE	30833.33333
BBA	20000
ETE	16100

7.4 Multi-Column GROUP BY

Grouping on multiple columns produces one row per unique combination of those columns:

```
-- sql  
select gender, type, sum(basic_salary) as Total_sal  
from employee  
group by gender, type;
```

Output table:

gender	type	Total_sal
M	T	107000
F	T	12000
F	O	10000

7.5 WHERE vs HAVING

This is one of the most important performance concepts in SQL. The two clauses filter at completely different execution phases:

Clause	Phase	Filters On
WHERE	Pre-aggregation	Individual rows — eliminates rows BEFORE the GROUP BY phase
HAVING	Post-aggregation	Group summaries — eliminates entire groups AFTER aggregation

HAVING: departments with average total income above 15,000

```
-- sql
select d.name, avg(e.basic_salary + e.others) as avg_sal
from employee e, department d
where e.dept_id = d.dept_id
group by d.name
having avg(e.basic_salary + e.others) > 15000;
```

Output table:

name	avg_sal
CSE	30833.33333
BBA	20000
ETE	16100

Note: You CANNOT use HAVING to filter on non-aggregated columns. HAVING is specifically for conditions involving aggregate functions (COUNT, AVG, SUM, etc.). Use WHERE for everything else.

7.6 Subqueries Inside HAVING

Departments whose average total pay exceeds the company-wide salary mean

```
-- sql
select d.name, avg(e.basic_salary + e.others) as avg_sal
from employee e, department d
where e.dept_id = d.dept_id
group by d.name
having avg(e.basic_salary + e.others) > (
    select avg(basic_salary) from employee -- dynamic company-wide baseline
);
```

Output table:

name	avg_sal
CSE	30833.33333
BBA	20000

The inner subquery computes 18,000.00 (global average basic salary). The outer HAVING clause then discards departments whose combined average does not exceed this benchmark. Since the benchmark is computed live from the data, the query remains self-correcting as data changes.

Finding the department with the highest average salary (subquery workaround)

```
-- sql
-- MySQL does not support max(avg(...)) directly,
-- so we use a derived table (subquery in FROM)
SELECT max(avg_salary)
FROM (
    SELECT avg(basic_salary) AS avg_salary
    FROM employee
    GROUP BY dept_id
) AS dept_avg;
```

Output : 23333.333333

7.7 Handling Departments with Zero Employees

All departments including ICT (no staff) — using LEFT JOIN + IFNULL

```
-- sql
select b.name, ifnull(a.avg_sal, 0) as avg_salary from
(select d.name as dept_name, avg(e.basic_salary) as avg_sal
from department d
left join employee e on d.dept_id = e.dept_id
group by d.dept_id) a, (select dept_id, name from department) b
where a.dept_id = b.dept_id;
```

Output table:

name	avg_salary
CSE	23333.33333
BBA	16000
ETE	13500
ICT	0

8. Verified Database State Summary

The following table records verified output metrics from live query execution on the populated the database:

Metric	Value	Query Used
Total Employee Count	7 Active Rows	SELECT COUNT(*) FROM Employee
Maximum Name Length	6 Characters ('Arshad')	SELECT MAX(LENGTH(Name))

Global Average Basic Salary	18,000.00 BDT	SELECT AVG(Basic_salary)
Dept 3 Average Basic Salary	13,500.00 BDT	...WHERE Dept_Id = 3
Total Research Papers	42 Papers	SELECT SUM(Papers) FROM Teacher_Details
Distinct Active Departments	3 (CSE, BBA, ETE)	SELECT COUNT(DISTINCT Dept_Id)
Course Enrollments (new table)	4 Valid Records	SELECT COUNT(*) FROM Course_Enrollment

9. Conclusion

This laboratory series demonstrates the systematic process of building a reliable relational database from scratch using MySQL. The progression follows a logical order: first establishing clean structural schemas with DDL, then enforcing business rules through layered constraints, then manipulating and querying data using DML.

Key Takeaways

- DDL constraints (PRIMARY KEY, CHECK, FOREIGN KEY, UNIQUE) shift validation responsibility from the application code to the database engine, ensuring data quality at the storage boundary regardless of how data is inserted.
- Constraint violations produce precise, descriptive error codes (1062, 3819, 1452) that identify exactly which rule was broken and on which table.
- ALTER TABLE allows schema evolution without data loss an essential capability for production systems.
- Built-in functions enable powerful data transformation without external processing.
- The WHERE / HAVING distinction is critical for both query correctness and performance: pre-aggregate filtering reduces the dataset before grouping; post-aggregate filtering eliminates groups after summarisation.
- Subqueries and LEFT JOINS combined with IFNULL enable dynamic, self-referential, and completeness-aware reporting pipelines.